OutdoorMic: An End-to-End System for Outdoor Bark Event Detection and Analysis

J. Mesches
jmesches@gmail.com

April 2026

Abstract

OutdoorMic is a self-hosted monitoring system that continuously listens for dog barks on an exterior microphone, records each event, classifies it with a convolutional neural network, estimates the number of distinct vocalizations per clip via a GPU onset counter, and aggregates the results against the Fort Collins noise ordinance (§ 4-94). This report describes the hardware setup, the real-time capture pipeline, the labeled dataset, the CNN architecture and training procedure, offline validation, and the reporting workflow. Over a 13-day evaluation window the trained classifier reached 95.6 % out-of-fold accuracy on 702 labeled clips, and scored 2,278 detector events (1933 classified as barks, 5808 estimated individual barks). All thirteen days exceeded one or more of the heuristic thresholds derived from § 4-94.

1 Introduction

Persistent outdoor barking is a common source of residential noise complaints but is difficult to document: human observers do not scale, and commercial decibel meters capture sound pressure without regard to source identity. OutdoorMic aims to bridge this gap with an always-on pipeline that (i) triggers on acoustic events in the frequency band characteristic of domestic dog barks, (ii) classifies each triggered clip with a small CNN trained on hand-labeled recordings, (iii) estimates the number of individual barks per clip, and (iv) summarizes the activity against the qualitative criteria of the Fort Collins noise ordinance. The full system — hardware, firmware, classifier, and reporting — runs on a single Linux host.

2 Hardware Setup

Microphone. A consumer-grade lavalier-style electret microphone is mounted outdoors and routed indoors through a 6 m shielded cable to the host's analog microphone input. The input is driven by the host's on-board Realtek ALC897 codec, exposed to Linux through ALSA/PipeWire as the device `alsa_input.pci-0000_00_1f.3.analog-stereo`.

Analog gain. Realistic outdoor sounds are quiet. To keep the codec's quantization noise floor well below the target signal, three gain stages are cascaded before digitization:

Stage	ALSA control	Setting
Capture level	`Capture`	`rec_level=49`
Mic preamp	`Mic Boost`	`mic_boost=1`
Digital trim	software	19 dB

The digital trim is applied in floating point after sample acquisition so that it does not clip inside the codec. Values are persisted in `config.toml` (see Listing 1) and re-applied by the detector at startup.

Host. Capture, scoring, training and reporting all run on the same Linux workstation (`Linux 6.17`, Python 3.12) with an NVIDIA RTX 5070 Ti GPU for PyTorch work. Network egress is limited to the local MQTT and InfluxDB endpoints; no external services are contacted.

3 Data Collection Pipeline

3.1 Real-time detector

The long-running process `bark_detector.py` streams audio from the input at 48 kHz, 4096-sample chunks (≈85 ms). Each chunk is scaled by the digital-gain factor, filtered through a cascaded biquad band-pass (SOS form, `scipy.signal.butter`) tuned to the dominant acoustic energy of the dog being monitored, then reduced to a full-scale RMS:

$$\mathrm{RMS}_{\mathrm{dBFS}}(x) = 20\log_{10}\left(\sqrt{\frac{1}{N}\sum_n x_n^2} + 10^{-12}\right). \quad (1)$$

When $\mathrm{RMS}_{\mathrm{dBFS}} \geq -30$ dBFS the detector enters the RECORDING state, continues writing samples from a rolling pre-roll ring buffer, and captures audio until the RMS falls below threshold for a full 0.2 s cooldown. Events shorter than 0.1 s are discarded; events exceeding 3.0 s are closed and a new event begins on the next excursion.

3.2 Per-event outputs

Each closed event produces three artifacts:

Listing 1: `config.toml` (detector).

```
[audio]
sample_rate = 48000
chunk = 4096
[audio.alsa]
rec_level = 49
mic_boost = 1
[audio.gain]
digital_db = 19

filter = [
  { type="bandpass", enabled=true,
    low_hz=600.0, high_hz=800.0, order=12 },
]

[detection]
threshold_dbfs = -30.0
pre_roll_s = 1.0
cooldown_s = 0.2
max_event_s = 3.0
min_event_s = 0.1
```

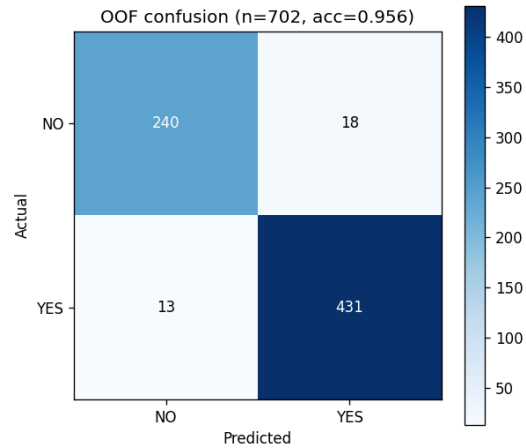


Figure 1: Out-of-fold confusion matrix aggregated over all five folds. Accuracy: 95.6 %. Rows are ground truth, columns are predictions.

1. A 48-kHz, mono, 16-bit PCM WAV file named `bark_YYYYMMDD_HHMMSS.wav` (UTC), written to `recordings/`.
2. One MQTT message on `homeassistant/mic/dog` carrying the UTC timestamp, peak RMS in dBFS, estimated bark count, duration, and the relative WAV path. Home Assistant auto-discovery sensors make the series available as entities.
3. One InfluxDB point in the `bark_events` bucket (measurement `bark_event`), tagged with the same fields for long-term time-series storage.

The process is run under the user-level systemd unit `bark-detector.service` with `Restart=on-failure` so that a crash in any single component does not produce a data gap of more than a few seconds.

4 Dataset and Labeling

The labeling tool `label_barks.py` is a Tkinter UI that steps through unlabeled recordings in `recordings/`, plays each clip, and accepts YES (real bark), NO (false positive), or EXCLUDE (skip) via single keystrokes. Labels are persisted in `bark_labels.xlsx` alongside the filename, peak RMS, and MQTT timestamp. At the time of this report the workbook contains 702 labeled clips — 444 YES and 258 NO — drawn from multiple days of production capture.

Class imbalance. The detector is tuned conservatively (−30 dBFS, 600–800 Hz), so most triggered clips are genuine. The residual negatives are mostly wind, traffic noise, and nearby voices; these are kept in the training set to stabilize the decision boundary near the threshold.

5 Model Architecture

`BarkCNN` is a compact 2D convolutional network designed to fit a small labeled corpus without overfitting. Input features are 64-band log-mel spectrograms (`n_fft` = 1024, hop = 256, $f_{\min} = 50$ Hz, $f_{\max} = 20$ kHz) of 3 s clips, power-to-dB with `ref=max`, standardized against the global training mean/std.

Stage	Ch.	Notes
Conv3×3 + BN + ReLU + MaxPool	16	
Conv3×3 + BN + ReLU + MaxPool	32	
Conv3×3 + BN + ReLU + MaxPool	64	
Conv3×3 + BN + ReLU	64	
AdaptiveAvgPool → 1×1	64	
Dropout(0.3) + Linear(→1)	1	BCE logit

The network has approximately 60k parameters and runs on CPU in under 1 ms per clip.

6 Training

Training is performed on GPU with features pre-materialized in VRAM for speed. Five-fold stratified CV (`StratifiedKFold`, seed 42) is used for the held-out confusion matrix; a final model is trained on all 702 clips for production inference.

Optimization. `AdamW` (lr = 10^{-3}, wd = 10^{-4}) with `CosineAnnealingLR`, `BCEWithLogitsLoss`, and a per-fold `pos_weight` set to $n_{\text{neg}}/n_{\text{pos}}$ to counter class imbalance. 60 epochs per fold with early-stopping patience of 12, batch size 64.

Augmentation. On-GPU SpecAugment with two frequency masks (up to $n_{\text{mels}}/8$ each), two time masks (up to $T/10$ each), and time-shift roll with uniform offset in $[-T/10, T/10]$. Each mask is applied with probability 0.5.

7 Validation

Per-fold accuracy (Table 1) ranges from 0.929 to 0.986. The aggregate out-of-fold confusion matrix (Figure 1)

Table 1: Per-fold held-out accuracy and confusion matrices.

Fold	Acc.	TN	FP	FN / TP
1	0.957	49	3	3 / 86
2	0.965	48	4	1 / 88
3	0.986	51	1	1 / 87
4	0.943	46	5	3 / 86
5	0.929	46	5	5 / 84

yields precision/recall of 0.960 / 0.971 for the YES class and 0.949 / 0.930 for the NO class (F1 0.965 and 0.939 respectively). Because the downstream ordinance heuristics are integrated over hours and days, the per-clip error floor of ∼4.4 % contributes at most a few units of bias to hourly bark counts, well below the thresholds used below.

8 Scoring Pipeline

The offline scorer (`bark_report.py`) re-processes every recording written by the detector. Rather than reusing the `librosa` feature extractor from training, it replicates the log-mel computation on GPU with `torchaudio.transforms.MelSpectrogram` and batched `power-to-dB(ref=max)`. This is numerically equivalent to the training-time librosa pipeline to within $\sim 10^{-3}$ dB and reduces end-to-end scoring time for ∼5400 clips from ∼90 s to under 7 s on the RTX 5070 Ti.

Parallel I/O. WAV decoding is parallelized across a `ProcessPoolExecutor` of `n_cpu` workers using `soundfile.read`. Workers hand fixed-length `float32` arrays to the main process, which stacks them into batches of 256 and hands them to the GPU. `tqdm` wraps the worker output for a live progress bar.

Per-clip bark counter. A single event can contain between one and four distinct barks. To estimate the number of barks per clip the scorer runs a GPU onset counter directly on the raw waveform:

1. Compute a 20 ms RMS envelope in dB (`F.avg_pool1d` of x^2).
2. Non-max suppress with a ±150 ms window via `F.max_pool1d`.
3. Keep peaks that are (a) within 12 dB of the clip's loudest frame and (b) above an absolute floor of −45 dBFS.
4. Clip the per-clip count at 8 to guard against buzzing.

For clips whose classifier probability is below threshold the estimated count is forced to zero; for clips above threshold it is $\max(\text{count}, 1)$. Across the evaluation window the mean multiplier is 3.00 barks per bark-clip (5808/1933).

9 Analysis Framework

Filenames record UTC. The reporter converts each timestamp to `America/Denver` local time via `zoneinfo` before any aggregation so that hourly statistics align with the local day. Events with classifier probability below 0.10 are dropped from the aggregates to prevent low-confidence detector triggers from dominating the bar charts; events on explicitly excluded dates are also removed.

Ordinance heuristics. Fort Collins § 4-94 defines "unreasonable noise" qualitatively via (i) time of day, (ii) duration, (iii) noise level, and (iv) other disruptive factors. It prescribes no numeric thresholds and requires a citizen affidavit. The reporter applies the following configurable heuristics to flag candidate violations:

Heuristic	Threshold
Night hours	22:00 – 07:00 local
Day hourly level	≥ 20 barks/hr
Night hourly level	≥ 5 barks/hr
Sustained duration	≥ 3 consecutive hrs @ 5+/hr
Daily total	≥ 100 barks

These thresholds are defaults exposed as CLI flags; they are not a legal finding.

10 Results

Across the 13-day evaluation window (April 8 – 20, 2026, local) the scorer processed 5385 raw detector events. 2278 survived the minimum-confidence filter; of these 1933 were classified as barks, and the GPU onset counter estimated 5808 distinct barks in total. All 13 days exceeded one or more of the ordinance heuristics.

The hourly timeline (Figure 2) shows strong concentration during daylight hours with a conspicuous weekend peak. Aggregating over days (Figure 3A) localizes the peak of the day between 14:00 and 18:00 local. The day-of-week × hour view (Figure 3B) isolates Sunday 17:00 as the worst cell: 502 estimated barks.

The full date × hour heatmap (Figure 4) visualizes which cells exceed the heuristics. The worst day in the window is Sunday April 19 with 1314 estimated barks across 11 flagged hourly cells; the quietest flagged day (Thursday April 9) still exceeds the day-hourly-level heuristic.

11 Limitations

Single-dog assumption. The classifier is trained on one dog's bark captured through one microphone. Recalibrating for a different setting requires re-labeling and, likely, retuning the band-pass filter.

Bark-count heuristics. The GPU onset counter operates on the raw waveform, not on source-separated audio. In overlapping events (a bark during a car horn, for example) it may over-count; during wind gusts of similar spectral shape it may fire spuriously on waveform

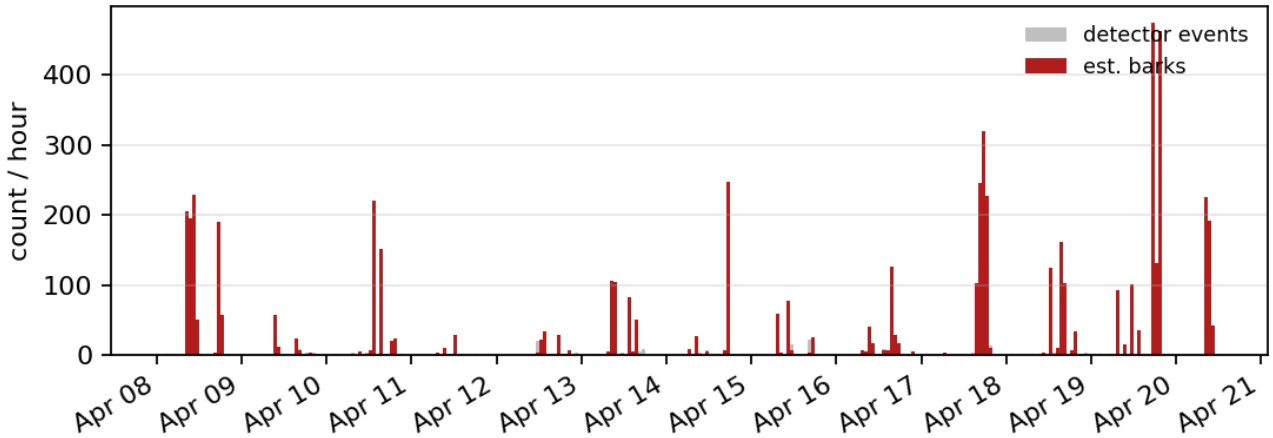


Figure 2: Hourly bark activity over the full evaluation window, local time (`America/Denver`). Grey: detector events; red: estimated barks. Each tick is one local midnight.

peaks that the classifier subsequently suppresses only when the full clip is considered.

Ordinance mapping. § 4-94 is adjudicated subjectively and requires a sworn citizen affidavit. The heuristics here are a quantitative proxy for the qualitative factors the ordinance lists; they are not a legal finding and are not presented as evidence.

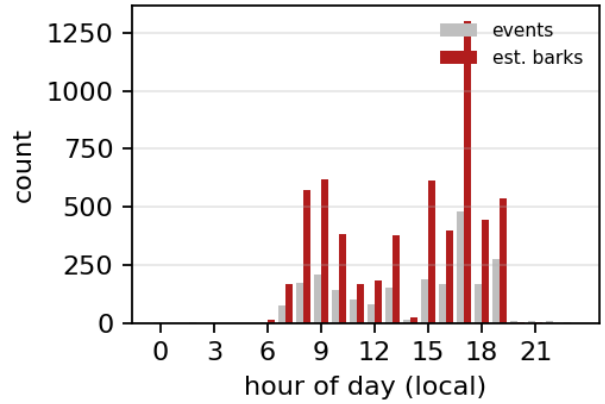
Clock. Filenames encode UTC and conversion to local time crosses the DST boundary cleanly, but any hours missing from the detector log (service restart, power outage) will appear as zero in the aggregates rather than as an explicit gap marker.

12 Reproducibility

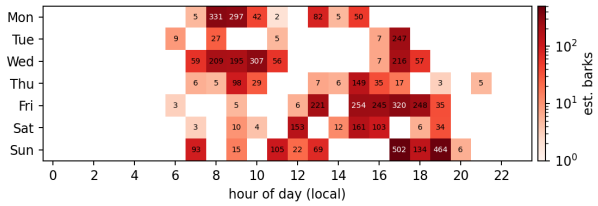
All scripts, configuration, and labeled data live in a single self-contained directory. The relevant entry points are:

- `bark_detector.py` — capture daemon (systemd-managed).
- `label_barks.py` — labeling UI.
- `train_bark_cnn.py` — CV training and production fit.
- `bark_report.py` — offline scoring and analysis.
- `report_figures.py` — generator for this report's figures.

Dependencies are pinned in the local `venv` and include `torch 2.11`, `torchaudio 2.11`, `librosa 0.11`, `scikit-learn 1.8`, and `paho-mqtt`.



(a) Totals by hour of day.



(b) Day-of-week × hour heatmap (log color scale).

Figure 3: Hour-of-day aggregates. (A) shows detector events (grey) versus estimated barks (red). (B) shows estimated barks only.

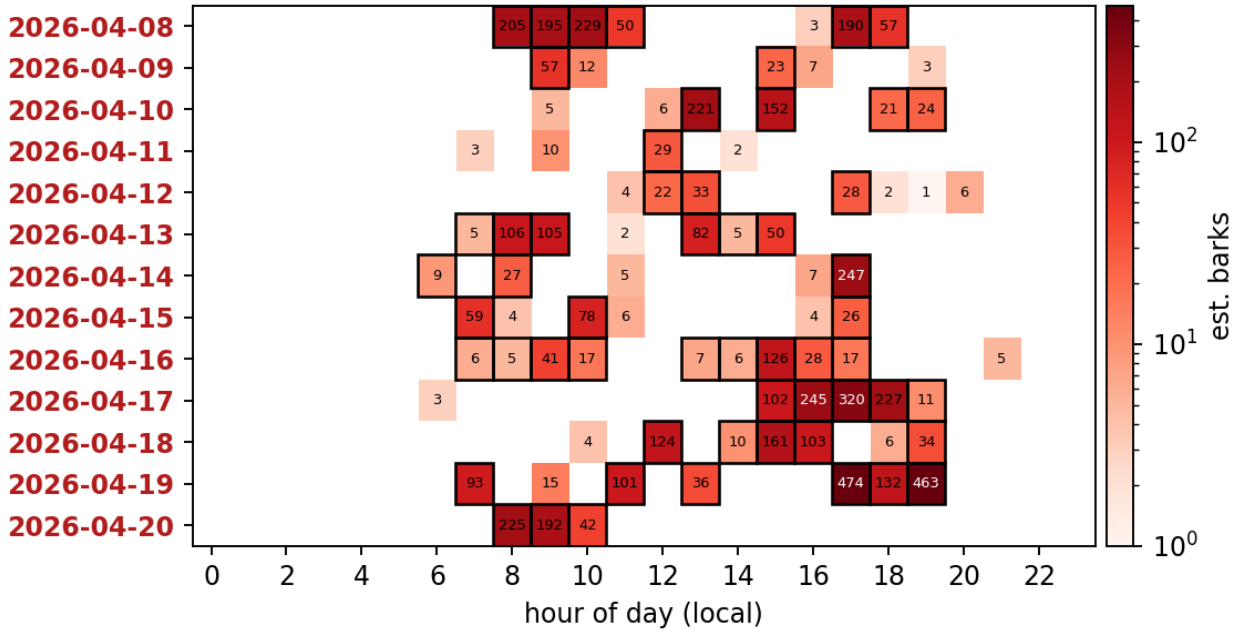


Figure 4: Estimated barks per (date, hour) cell. Color: log-scaled bark count. Black outlines: cells flagged under the § 4-94 heuristics. Red bold y-axis labels: dates flagged at the day level.